

How to Scale Your Model

Chapter 1: All About Rooflines — Notes & Exercises

Akarsh Upadhyay

October 2026

Introduction

What this book should let you do:

- Ballpark how close parts of your model are to their theoretical optimum.
- Make informed choices about different parallelism schemes at different scales.
- Estimate the cost and time required to train a large Transformer model.
- Design algorithms that take advantage of specific hardware affordances.
- Design hardware driven by an explicit understanding of what limits current algorithm performance.

Questions to answer:

- How long should a matrix multiply of a certain size take? At what point is it compute-bound vs. memory-bound?
- How are TPUs wired together to form training clusters? How much bandwidth does each part of the system have?
- How long does it take to gather, scatter, or re-distribute arrays across multiple TPUs?
- How do we efficiently multiply matrices that are distributed differently across devices?

1 Intro to Rooflines

When we run an algorithm on hardware, we are bounded by three things:

1. how fast the computer can do math (FLOPs/s),
2. the bandwidth available for moving data (bytes/s),
3. the total memory available to store data (bytes).

Computation.

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/s}}$$

e.g. H100: $\approx 9.89 \times 10^{14}$ bf16 FLOPs/s (dense).

Communication. Happens both *within* a chip (HBM $\leftrightarrow$ compute) and *between* chips (network).

$$T_{\text{comms}} = \frac{\text{Communication bytes}}{\text{Network/memory bandwidth (bytes/s)}}$$

Bounds on runtime. Computation can typically be overlapped with communication (within a chip and between chips), so

$$T_{\text{lower}} = \max(T_{\text{math}}, T_{\text{comms}}), \quad T_{\text{upper}} = T_{\text{math}} + T_{\text{comms}}.$$

- $T_{\text{math}} > T_{\text{comms}} \Rightarrow$ **compute-bound**.
- $T_{\text{comms}} > T_{\text{math}} \Rightarrow$ **memory/communication-bound**.

Arithmetic intensity

The ratio of total FLOPs an algorithm performs to the number of bytes it needs to communicate:

$$\text{AI} = \frac{\text{Computation FLOPs}}{\text{Communication bytes}} \quad [\text{FLOPs/byte}]$$

It is a property of the *algorithm*, not a speed. The hardware has its own critical intensity $\text{AI}_{\text{hw}} = \text{peak FLOPs/s} / \text{bandwidth}$. We are compute-bound iff $\text{AI} > \text{AI}_{\text{hw}}$.

Example 1: dot product

$x, y \in \mathbb{R}^N$ (bf16), $z = x \cdot y \in \mathbb{R}$.

$$\text{Bytes loaded} = 2(N + N) = 4N, \quad \text{bytes written} = 2,$$

$$\text{FLOPs} = \underbrace{N}_{\text{mults}} + \underbrace{N-1}_{\text{adds}} = 2N - 1 \approx 2N,$$

$$\text{AI} = \frac{2N-1}{4N+2} \xrightarrow{N \rightarrow \infty} \frac{1}{2}.$$

Far below any accelerator's critical intensity, so a dot product is always memory-bound.

1.1 Visualizing rooflines

How do you visualize the tradeoff between memory and compute? How do FLOPs/s correlate with arithmetic intensity?

Let F = total FLOPs, B = total bytes moved, T = total runtime. Achieved throughput is F/T :

$$T \approx \max(T_{\text{math}}, T_{\text{comms}}) = \max\left(\frac{F}{\text{peak}}, \frac{B}{\text{BW}}\right) = \max\left(\frac{F}{\text{peak}}, \frac{F}{\text{AI} \cdot \text{BW}}\right) \quad (B = F/\text{AI})$$

$$= F \cdot \max\left(\frac{1}{\text{peak}}, \frac{1}{\text{AI} \cdot \text{BW}}\right)$$

$$\frac{F}{T} = \frac{1}{\max\left(\frac{1}{\text{peak}}, \frac{1}{\text{AI} \cdot \text{BW}}\right)}$$

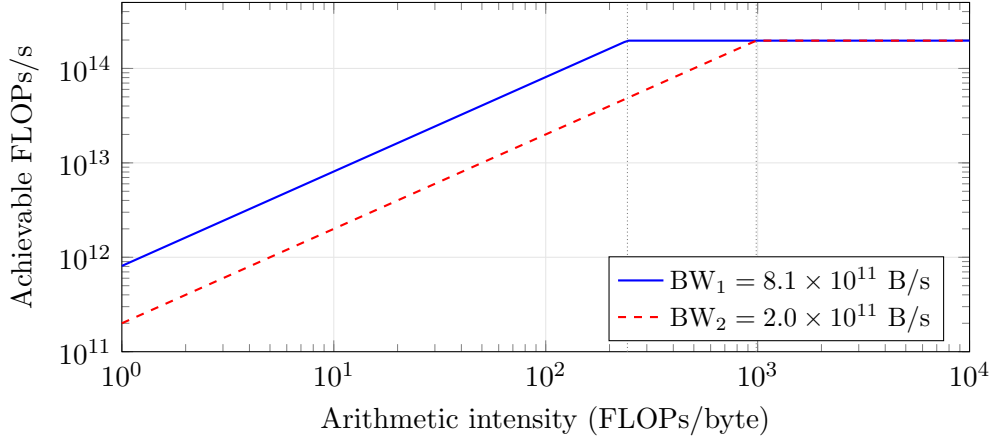
Using $\frac{1}{\max(1/a, 1/b)} = \min(a, b)$:

Roofline

$$\text{achievable FLOPs/s} = \min(\text{peak FLOPs/s}, \text{AI} \cdot \text{BW})$$

Correction

Taking the reciprocal flips **max** into **min** (not **max**, and not $1/\text{min}$). Sanity check: throughput can never exceed peak, so it must be a min. Also, T is the *total* runtime, not just the time to move data.



Slanted part: memory-bound, slope = bandwidth. Flat part: compute-bound (the “roof”). Corner at $\text{AI} = \text{peak}/\text{BW}$. Higher bandwidth moves the corner left; between the two corners an algorithm is compute-bound only on the faster-memory system.

What question can I get an answer to by following this plot?

Given my algorithm’s arithmetic intensity, **what fraction of peak can I actually get on this hardware**, and is the fix

- more bandwidth,
- more compute, or
- restructuring the algorithm to raise intensity?

1.2 Matrix multiplication

$X \in \mathbb{R}^{B \times D}$, $Y \in \mathbb{R}^{D \times F}$, $Z = XY \in \mathbb{R}^{B \times F}$, all bf16.

$$\begin{aligned} \text{Bytes} &= \underbrace{2(BD + DF)}_{\text{read}} + \underbrace{2BF}_{\text{write}}, \\ \text{FLOPs} &= B(D + D - 1)F = B(2D - 1)F \approx 2BDF, \\ \text{AI} &= \frac{2BDF}{2(BD + DF + BF)} = \frac{BDF}{BD + DF + BF}. \end{aligned}$$

If $B \ll D, F$, then $\boxed{\text{AI} \approx B}$. Compute-bound when $\text{AI}_{\text{matmul}} > \text{AI}_{\text{accel}}$; for a TPU v5e this is

$$\boxed{B > 240}.$$

1.3 Network rooflines

Split the matmul across two chips along D : each chip holds $X[:, D/2]$ and $Y[D/2, :]$, computes a partial Z , then the partial results are aggregated over the inter-chip link.

$$\begin{aligned} T_{\text{math}} (\text{per chip}) &= \frac{2B \frac{D}{2} F}{\text{peak FLOPs/s}} = \frac{BDF}{\text{peak FLOPs/s}}, \\ T_{\text{comms}} (\text{per chip}) &= \frac{2BF}{\text{ICI bandwidth}} \quad (\text{send the bf16 partial sum } \in \mathbb{R}^{B \times F}). \end{aligned}$$

So $\text{AI} = \frac{BDF}{2BF} = \frac{D}{2}$: the network roofline depends on D , not B . Compute-bound when $D/2 > \text{peak}/\text{ICI BW}$.

Correction

My handwritten version mixed the HBM load bytes ($BD + DF + 2BF$) into the network term. For the *network* roofline only the bytes crossing the chip-to-chip link count; HBM traffic is a separate roofline on each chip.

2 Worked Problems

Exercise 1.1 int8 matmul

$X \in \mathbb{R}^{B \times D}$, $Y \in \mathbb{R}^{D \times F}$, $Z \in \mathbb{R}^{B \times F}$, all int8 (1 byte). HBM bandwidth 8.1×10^{11} B/s, int8 peak 3.94×10^{14} OPs/s.

1. Bytes loaded = $BD + DF$; written back = BF .
2. Total OPs = $B(2D - 1)F \approx 2BDF$.
3. $\text{AI} = \frac{2BDF}{BD + DF + BF} \approx 2B$ for $B \ll D, F$. Compute-bound when

$$2B > \frac{3.94 \times 10^{14}}{8.1 \times 10^{11}} \approx 486 \implies B > 243.$$

4. Roofline estimates:

$$T_{\text{math}} = \frac{2BDF}{3.94 \times 10^{14}}, \quad T_{\text{comms}} = \frac{BD + DF + BF}{8.1 \times 10^{11}},$$

$$\text{lower bound} = \max(T_{\text{math}}, T_{\text{comms}}), \quad \text{upper bound} = T_{\text{math}} + T_{\text{comms}}.$$

For small B , $T_{\text{comms}} \approx DF/8.1 \times 10^{11}$.

Exercise 1.2 int8 weights + bf16 activations

Weights in low precision (int8); activations and compute in higher precision (bf16): $\text{bfloat16}[B, D] * \text{int8}[D, F] \rightarrow \text{bfloat16}[B, F]$. Assume 1.97×10^{14} bf16 FLOPs/s. At what batch size do we become compute-bound?

$$T_{\text{math}} = \frac{2BDF}{\text{peak}}, \quad T_{\text{comms}} = \frac{2BD + DF + 2BF}{\text{BW}}.$$

Compute-bound when $T_{\text{math}} \geq T_{\text{comms}}$. With $DF \gg 2BD, 2BF$:

$$\frac{2BDF}{\text{peak}} \geq \frac{DF}{\text{BW}} \implies B \geq \frac{1}{2} \cdot \frac{\text{peak}}{\text{BW}} = \frac{1}{2} \cdot \frac{1.97 \times 10^{14}}{8.1 \times 10^{11}} \approx \boxed{120}.$$

Half the bf16 threshold of ~ 240 , because weights now cost 1 byte instead of 2.

Exercise 1.3

For the problem above, make a roofline plot of peak FLOPs/s vs. B for several values of D and F .

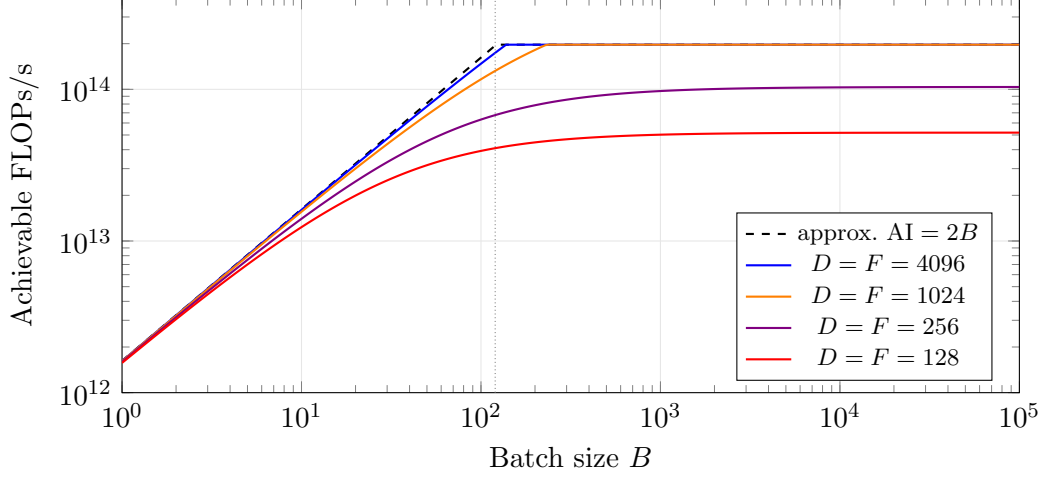
Exact intensity and roofline:

$$\text{AI}(B) = \frac{2BDF}{DF + 2BD + 2BF}, \quad \text{FLOPs/s} = \min(\text{peak}, \text{AI}(B) \cdot \text{BW}).$$

- **Assumption** $2B(D + F) \ll DF$: $\text{AI} \approx 2B$, independent of D and F . Linear rise, then flat at peak from $B \approx 120$.

- **Exact:** as $B \rightarrow \infty$, $AI \rightarrow \frac{DF}{D+F}$. If this cap is below $\text{peak}/\text{BW} \approx 243$, the matmul *never* becomes compute-bound (e.g. $D = F = 128$ caps at 64).

Except for the approximate curve, the exact curves for each (D, F) were first plotted with matplotlib; they are re-drawn below with pgfplots.



Exercise 1.4 per-batch weight matrices

$\text{int8}[B, D] *_D \text{int8}[B, D, F] \rightarrow \text{int8}[B, F]$, a different matrix for each batch element. What is the arithmetic intensity?

For each batch element: $[1, D] \times [D, F] \rightarrow [1, F]$, i.e. $2DF$ OPs; concatenate over the batch.

$$\text{Bytes} = \underbrace{BD}_{\text{load}} + \underbrace{BDF}_{\text{write}} + \underbrace{BF}_{\text{write}}, \quad \text{OPs} = B \cdot 2DF = 2BDF,$$

$$AI = \frac{2BDF}{BD + BDF + BF} \approx 2 \quad (BDF \gg BD + BF).$$

Peak int8 intensity = $3.94 \times 10^{14} / 8.1 \times 10^{11} \approx 486 \gg 2$, so this is **memory/comms-bound regardless of B** , reaching only $\sim 0.4\%$ of peak.

Correction

$AI = 2$ means *memory-bound*, not compute-bound. Intensity comes from reuse: in a normal matmul each weight byte is reused across B rows ($AI \approx 2B$); here each weight is used in exactly one multiply-add, so AI is stuck at 2. Same reason decode attention over per-sequence KV caches is memory-bound.

Exercise 1.5 memory rooflines for GPUs

Using the H100 spec sheet, at what batch size does a matmul become compute-bound? (Tensor Core FLOPs on the sheet are $2\times$ the true value, since they assume structured sparsity.)

bf16 throughout: $\text{FLOPs} = 2BDF$, $\text{bytes} = 2(BD + DF + BF)$. Spec: ~ 2000 TFLOPs/s with sparsity $\Rightarrow \sim 1000$ TFLOPs/s dense; HBM 3.35 TB/s.

$$T_{\text{math}} \geq T_{\text{comms}} : \quad \frac{2BDF}{\text{peak}} \geq \frac{2DF}{\text{BW}} \quad (B \ll D, F) \implies B \geq \frac{10^{15}}{3.35 \times 10^{12}} \approx \boxed{298}.$$

Correction

My notes used 3 TB/s and got $B \geq 333$. With the spec-sheet value of 3.35 TB/s the answer is ≈ 298 , close to the TPU v5e's ~ 240 .